

RAPPORT DE PROJET
DÉVELOPPEMENT JEE / ARCHITECTURE N-TIERS

SubTrack

Plateforme Intelligente de Gestion et
d'Optimisation d'Abonnements SaaS

Réalisé par :

EL HAIL Jaouad
KHOUKA Hajar

Encadré par :

Mme. Belmokaddem Houda

Année Universitaire : 2023 - 2024

Table des matières

1	Contexte, Problématique et Objectifs du Projet	2
1.1	Le Contexte Fonctionnel	2
1.2	La Problématique	2
1.3	Les Objectifs du Projet	2
2	Les Fonctionnalités Attendues	2
2.1	1. Espace Client (Gestion & Optimisation CRUD)	2
2.2	2. Automatisation (Zéro Saisie)	3
2.3	3. Espace Administrateur (Back-Office)	3
3	Les Acteurs et Utilisateurs du Système	3
3.1	Acteurs Humains	3
3.2	Acteurs Systèmes	4
4	Aspect Technique et Architecture du Système	4
4.1	Architecture N-Tiers (Jakarta EE)	4
4.2	Écosystème d'Automatisation et d'Intelligence Artificielle	4
5	Modélisation des Données (Schéma Relationnel)	5
5.1	Description des Entités Principales	5
6	Modélisation Fonctionnelle (Cas d'Utilisation)	6
6.1	Description des Acteurs et Fonctionnalités Principales	6
7	Modélisation Statique (Diagramme de Classes)	8
7.1	Description des Packages et Classes Principales	8
8	Architecture Logicielle (Diagramme N-Tiers)	9
8.1	Description des Couches et Responsabilités	9
9	Interfaces Principales de l'Application	11
9.1	Pilotage et Analyse des Données	11
9.2	Gestion du Catalogue SaaS	12
9.3	Configuration des Notifications et Gestion Profil	12

1 Contexte, Problématique et Objectifs du Projet

1.1 Le Contexte Fonctionnel

L'application **SubTrack** est une plateforme unifiée conçue pour aider les particuliers, les freelances et les entreprises (B2B) à centraliser, suivre et optimiser la gestion de leurs dépenses récurrentes en logiciels et services. Dans un monde où l'économie de l'abonnement est omniprésente, **SubTrack** agit comme un assistant financier intelligent, automatisant la collecte des factures et offrant une visibilité claire sur les engagements financiers continus.

1.2 La Problématique

Les utilisateurs et les entreprises perdent des centaines, voire des milliers de dirhams par an à cause d'abonnements oubliés, de renouvellements automatiques surprises, de hausses de prix silencieuses et de services redondants. Le suivi manuel via des tableurs est fastidieux, chronophage et sujet aux erreurs humaines. De plus, la gestion des abonnements internationaux implique des frais de conversion de devises qui complexifient le suivi du budget réel.

1.3 Les Objectifs du Projet

- Offrir une vue à 360 degrés de toutes les dépenses récurrentes via un tableau de bord analytique.
- Réduire les dépenses en identifiant proactivement les outils inutilisés ou redondants.
- Éliminer la saisie manuelle grâce à l'analyse intelligente des reçus par e-mail.
- Faciliter la gestion financière et comptable grâce à la centralisation et l'export des données.

2 Les Fonctionnalités Attendues

2.1 1. Espace Client (Gestion & Optimisation CRUD)

- **Gestion des abonnements (CRUD)** : Ajouter, modifier, suspendre ou résilier un abonnement avec gestion des fréquences (hebdomadaire, mensuelle, annuelle).
- **Enrichissement automatique** : Récupération des logos des services et catégorisation par tags personnalisés (ex : Loisirs, Travail, Cloud).

- **Tableau de bord & Analyse** : Affichage des coûts mensuels/annuels, prévisions budgétaires et suivi de l'historique pour détecter les hausses de prix.
- **Optimisation financière** :
 - Conversion automatique des devises étrangères en monnaie locale.
 - Détection algorithmique des abonnements redondants.
 - Centralisation des liens pour une résiliation en 1 clic.
- **Export comptable** : Génération d'archives de justificatifs pour la comptabilité (cible B2B/Freelance).
- **Système d'Alertes** : Notifications envoyées avant le prélèvement via Email, Telegram ou WhatsApp.

2.2 2. Automatisation (Zéro Saisie)

- **Synchronisation Email** : Connexion sécurisée aux boîtes mail pour détecter les nouvelles factures.
- **Extraction intelligente** : Lecture automatique des reçus pour en extraire le nom du service, le montant et la date d'échéance afin de créer ou mettre à jour l'abonnement sans intervention humaine.

2.3 3. Espace Administrateur (Back-Office)

- **Gestion des utilisateurs** : Administration des comptes clients.
- **Gestion du catalogue SaaS** : Maintien d'une base de données globale des services connus.
- **Supervision** : Suivi des logs systèmes et configuration globale.

3 Les Acteurs et Utilisateurs du Système

3.1 Acteurs Humains

- **Utilisateur B2C (Particuliers)** : Cherche à suivre ses dépenses personnelles (VOD, musique, box) pour maîtriser son budget.
- **Utilisateur B2B & Freelances** : A besoin de suivre ses licences logicielles professionnelles, d'éviter les doublons dans son équipe et d'exporter ses factures.
- **L'Administrateur** : Gère le back-office de **SubTrack**, maintient le système et assure le support.

3.2 Acteurs Systèmes

- **API Taux de Change** : Fournit les taux de conversion monétaire en temps réel.
- **Fournisseur Email** : Transmet les reçus au système via des webhooks.

4 Aspect Technique et Architecture du Système

Pour répondre aux exigences de robustesse et d'évolutivité, **SubTrack** repose sur une **architecture N-Tiers** stricte, basée sur l'écosystème Java/Jakarta EE, et intègre des micro-services externes pour l'intelligence artificielle.

4.1 Architecture N-Tiers (Jakarta EE)

Le système est divisé en couches indépendantes communiquant entre elles :

- **Couche Client (Navigateur Web)** : L'interface utilisateur est rendue en HTML/CSS/JS. Elle communique avec le serveur via des requêtes et réponses HTTP.
- **Couche Présentation (JSF - JavaServer Faces)** : Gère l'interface côté serveur à l'aide de pages *Facelets* (*.xhtml*). Ces pages sont liées dynamiquement (Data Binding) à des contrôleurs CDI appelés *Managed Beans*, qui interceptent les actions de l'utilisateur.
- **Couche Métier (Logique Applicative)** : Composée de *Services* (ex : `SubscriptionService`). Elle contient toute la logique métier de l'application (calculs, vérifications, règles de gestion) et fait le pont entre les contrôleurs et la base de données.
- **Couche Persistance (Hibernate / JPA)** : Assure la communication avec la base de données. Elle utilise le patron de conception DAO (*Data Access Object*) via des interfaces et leurs implémentations. Elle manipule des *Entités JPA* (`@Entity`) et génère les requêtes SQL/HQL grâce à l'`EntityManager`.
- **Base de Données Relationnelle** : Stocke de manière persistante les tables du système (utilisateurs, abonnements, historique, etc.).

4.2 Écosystème d'Automatisation et d'Intelligence Artificielle

En marge du cœur de l'application Jakarta EE, le système s'appuie sur une architecture événementielle pour l'automatisation :

- **n8n (Orchestrateur)** : Agit comme un middleware d'automatisation. Il écoute les webhooks (nouveaux emails reçus) et orchestre le flux de données.
- **API Google Gemini (LLM)** : Sollicité par l'orchestrateur n8n, ce modèle d'intelligence artificielle est chargé d'analyser le texte brut des emails pour en extraire des données structurées (JSON) qui sont ensuite injectées dans la base de données de SubTrack.

5 Modélisation des Données (Schéma Relationnel)

Afin de supporter les fonctionnalités de la plateforme **SubTrack**, la base de données relationnelle a été structurée autour d'entités clés garantissant l'intégrité, la traçabilité et l'optimisation des données. Le diagramme ci-dessous présente la structure des tables, leurs attributs, ainsi que les relations (Clés Primaires / Clés Étrangères) qui les unissent.

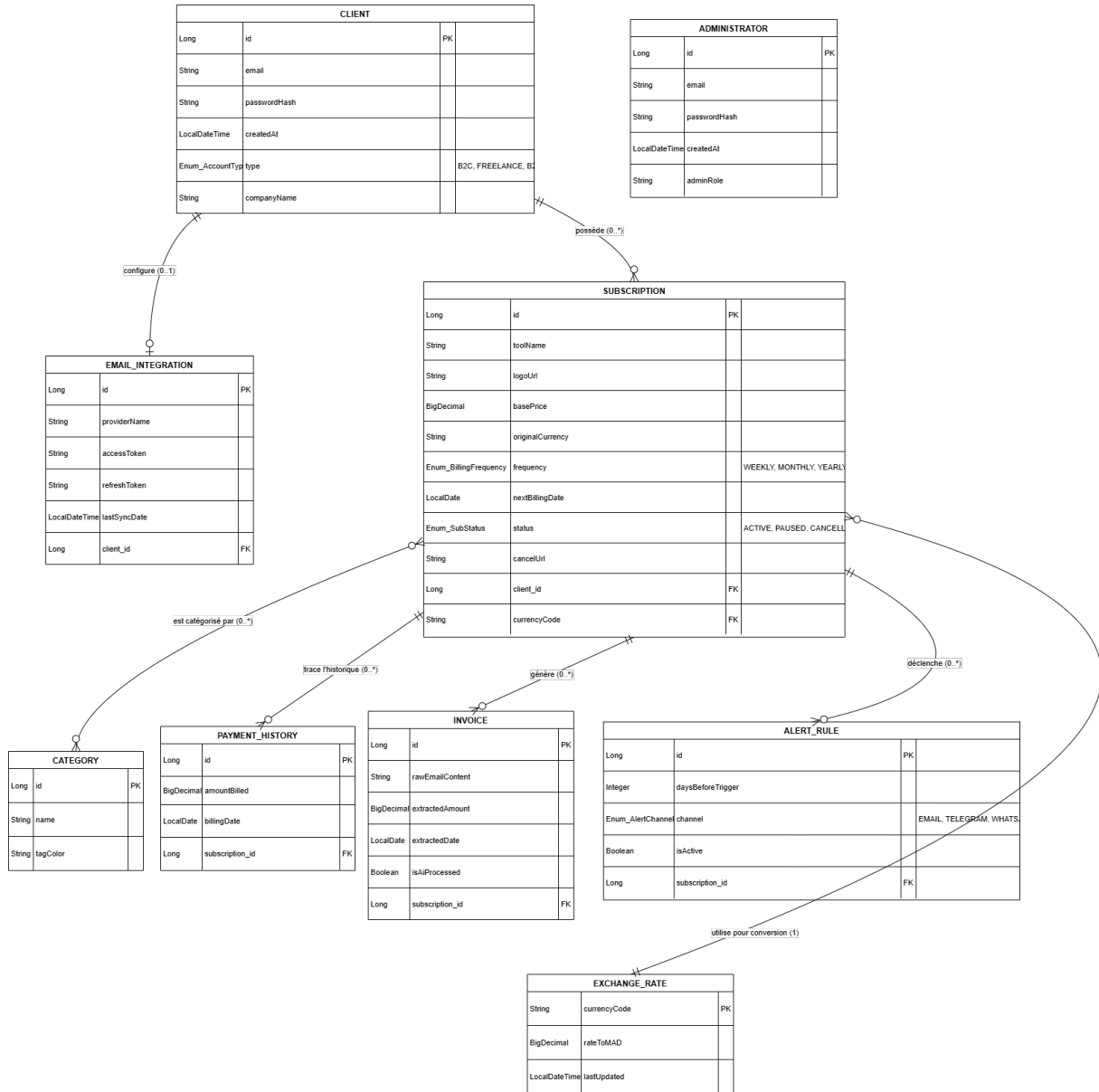


FIGURE 1 – Schéma Relationnel de la base de données SubTrack

5.1 Description des Entités Principales

L'architecture des données s'articule autour de plusieurs domaines fonctionnels :

- **Gestion des Utilisateurs** : Le système sépare logiquement les utilisateurs finaux (**CLIENT**) des administrateurs du système (**ADMINISTRATOR**). La table **CLIENT** gère

l'authentification et différencie les types de profils via l'énumération `AccountType` (B2C, Freelance, B2B).

- **Le Cœur Métier (Abonnements)** : La table `SUBSCRIPTION` est l'entité centrale du système. Un client peut posséder plusieurs abonnements (relation 1 : N). Cette table stocke les détails essentiels tels que le prix de base, la devise d'origine, la fréquence de facturation et le statut de l'abonnement (*Active*, *Paused*, *Cancelled*).
- **Suivi et Historique** : Afin de détecter les hausses de prix silencieuses, chaque abonnement est lié à la table `PAYMENT_HISTORY`, qui trace l'historique chronologique des montants réellement prélevés.
- **Automatisation et IA** :
 - `EMAIL_INTEGRATION` : Stocke les jetons d'accès sécurisés (OAuth) pour permettre au système de scanner les boîtes mail des clients (relation 0..1 avec `CLIENT`).
 - `INVOICE` : Conserve les données extraites par l'IA (Google Gemini) à partir des e-mails bruts, indiquant si la facture a bien été traitée (`isAiProcessed`).
- **Notifications et Alertes** : La table `ALERT_RULE` permet de définir de multiples règles de notification pour un même abonnement (ex : 3 jours avant via Email, 1 jour avant via WhatsApp).
- **Optimisation Financière** : La table `EXCHANGE_RATE` agit comme un référentiel mis à jour régulièrement, permettant de convertir instantanément les devises étrangères (`originalCurrency` de l'abonnement) vers la monnaie locale (MAD) pour les tableaux de bord. Enfin, la table `CATEGORY` permet de regrouper les abonnements par thématiques pour l'analyse budgétaire.

6 Modélisation Fonctionnelle (Cas d'Utilisation)

Afin de définir le périmètre fonctionnel et les interactions des différents acteurs avec la plateforme **SubTrack**, un diagramme de cas d'utilisation a été élaboré. Le schéma ci-dessous présente les acteurs principaux (humains et systèmes externes), les fonctionnalités offertes par le système, ainsi que les dépendances spécifiques entre ces actions (inclusions et extensions).

6.1 Description des Acteurs et Fonctionnalités Principales

Le périmètre fonctionnel du système s'articule autour de plusieurs types d'interactions et d'acteurs clés :

- **Les Acteurs Humains** : Le système sépare logiquement l'utilisateur final (*Utilisateur B2C / B2B*) de l'administrateur (*Administrateur*). L'utilisateur gère ses finances personnelles ou professionnelles, tandis que l'administrateur possède des

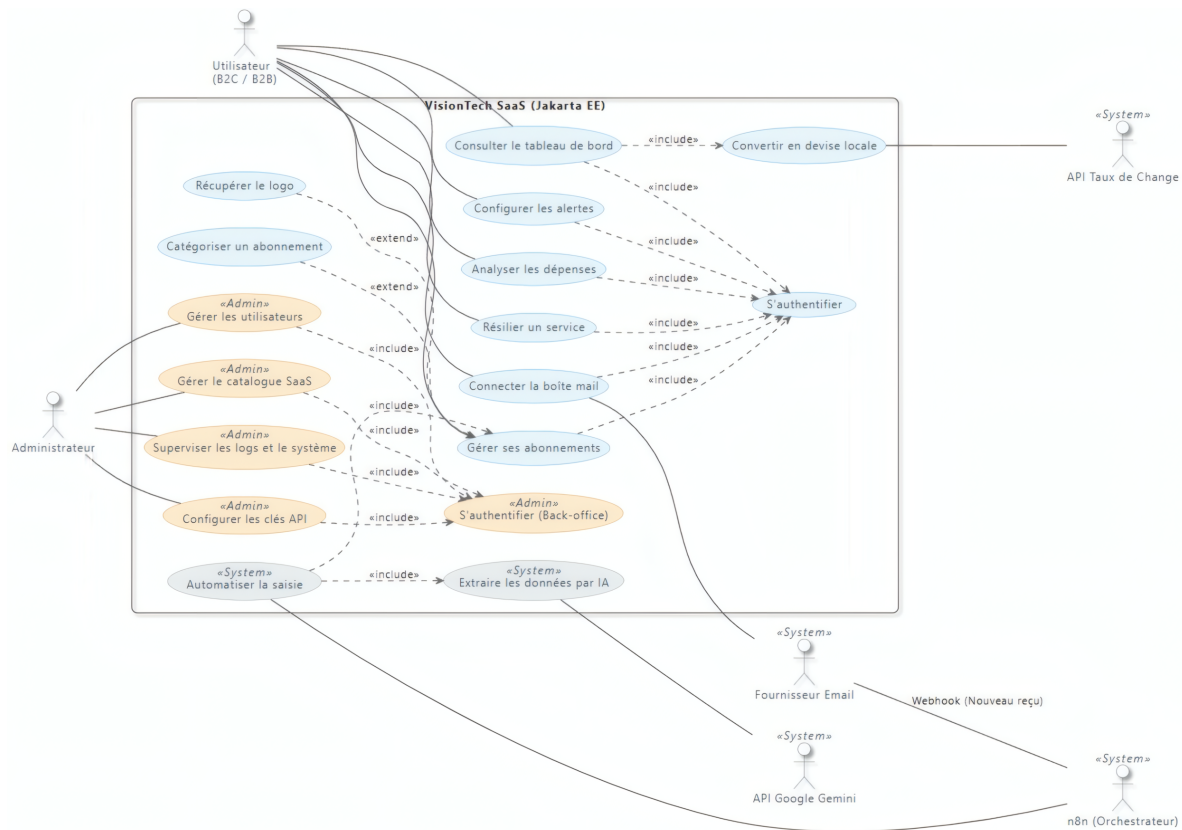


FIGURE 2 – Diagramme de Cas d'Utilisation du système SubTrack

privileges étendus pour maintenir le système, gérer le catalogue SaaS et superviser les logs.

- **L'Espace Utilisateur (Gestion et Analyse) :** L'utilisateur interagit avec le système pour gérer ses abonnements, configurer des alertes, connecter sa boîte mail et analyser ses dépenses via un tableau de bord. Toutes ces actions majeures nécessitent obligatoirement une authentification préalable (modélisée par la relation «include» vers "S'authentifier").
- **Fonctionnalités Optionnelles (Extensions) :** Lors de la gestion de ses abonnements, l'utilisateur a la possibilité d'enrichir ses données. Des actions optionnelles, telles que "Catégoriser un abonnement" ou "Récupérer le logo", viennent compléter le cas d'utilisation principal (modélisées par la relation «extend»).
- **L'Espace Administrateur (Back-Office) :** L'administrateur dispose de son propre accès sécurisé («include» vers "S'authentifier Back-office"). Ses cas d'utilisation spécifiques incluent la gestion des utilisateurs, la maintenance du catalogue SaaS global, ainsi que la configuration des clés API essentielles au système.
- **Les Acteurs Systèmes et l'Automatisation :** La plateforme interagit de manière autonome avec des systèmes externes pour automatiser les tâches :
 - *API Taux de Change* : Sollicitée automatiquement («include») lors de la consultation du tableau de bord pour convertir les montants en devise locale.

- *Fournisseur Email & n8n* : Lorsqu'un nouveau reçu est détecté par le fournisseur email (via Webhook), l'orchestrateur n8n déclenche le cas "Automatiser la saisie".
- *API Google Gemini* : Ce processus d'automatisation inclut obligatoirement («include») l'appel à l'IA Gemini pour extraire intelligemment les données financières de la facture.

7 Modélisation Statique (Diagramme de Classes)

Afin de concevoir l'architecture logicielle orientée objet de la plateforme **SubTrack**, un diagramme de classes UML a été élaboré. Le schéma ci-dessous présente l'organisation du code en packages logiques, les attributs et méthodes métiers des entités, ainsi que les relations (héritage, associations, multiplicités) qui les lient.

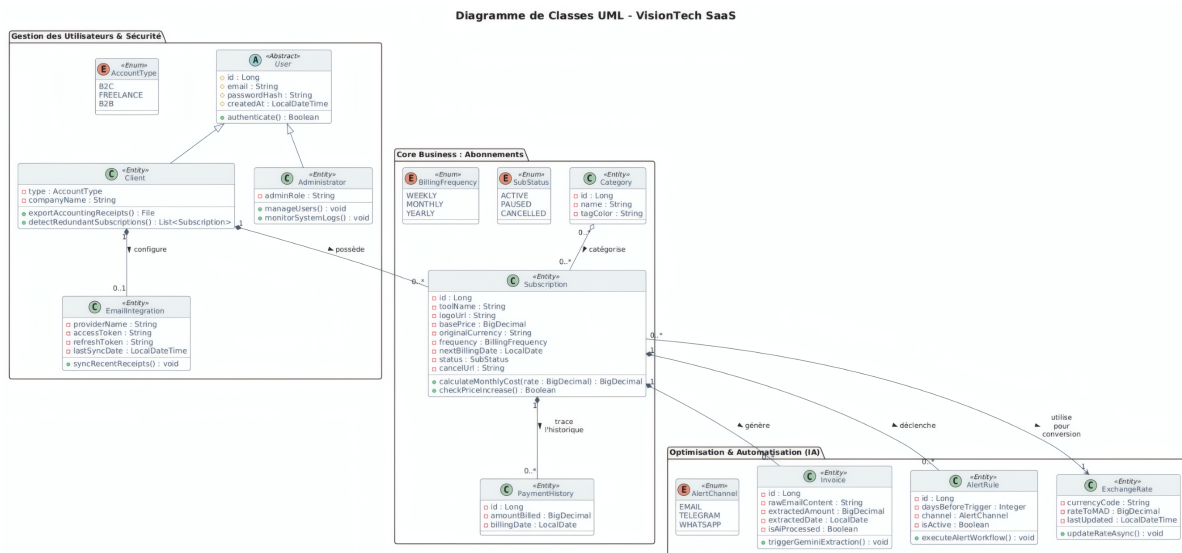


FIGURE 3 – Diagramme de Classes UML du système SubTrack

7.1 Description des Packages et Classes Principales

L'architecture applicative est découpée en trois domaines logiques (packages) :

- **Gestion des Utilisateurs & Sécurité** : Ce module exploite le principe d'héritage. Une classe abstraite **User** centralise les attributs communs d'authentification et la méthode `authenticate()`. Elle est étendue par deux entités concrètes : **Administrator** (qui possède les méthodes `manageUsers()` et `monitorSystemLogs()`) et **Client**. Le **Client** intègre une méthode clé d'optimisation (`detectRedundantSubscriptions()`) et peut configurer une instance d'**EmailIntegration** pour synchroniser ses reçus.
- **Core Business (Abonnements)** : La classe **Subscription** constitue le noyau métier du système. Un **Client** possède une ou plusieurs instances de cette classe.

Subscription encapsule non seulement les données, mais aussi la logique métier via des méthodes comme `calculateMonthlyCost()` et `checkPriceIncrease()`. Elle est liée à la classe **Category** pour le regroupement par thématiques, et génère des instances de **PaymentHistory** pour tracer l'évolution temporelle des facturations.

- **Optimisation & Automatisation (IA)** : Ce package regroupe les services avancés gravitant autour des abonnements :
 - **Invoice** : Modélise la facture traitée. Elle intègre la méthode `triggerGeminiExtraction()` qui indique l'interaction avec l'IA pour traiter le `rawEmailContent`.
 - **AlertRule** : Gère la logique de notification via la méthode `executeAlertWorkflow()`, en s'appuyant sur l'énumération **AlertChannel** (Email, Telegram, WhatsApp) pour définir le canal de communication.
 - **ExchangeRate** : Entité gérant la conversion de devises. Elle possède une méthode asynchrone `updateRateAsync()` garantissant que les calculs de coûts se font toujours sur des taux de change à jour.

8 Architecture Logicielle (Diagramme N-Tiers)

Afin de garantir une séparation claire des responsabilités et une maintenance évolutive de la plateforme **VisionTech**, une architecture distribuée en couches (N-Tiers) a été adoptée. Le schéma ci-dessous illustre le flux de données, depuis l'interaction de l'utilisateur sur le navigateur jusqu'à la persistance des données dans la base relationnelle.

8.1 Description des Couches et Responsabilités

L'écosystème technique repose sur l'intégration des frameworks Java EE (Jakarta EE), structuré en quatre niveaux principaux :

- **Navigateur Web (Client)** : Représente le point d'entrée pour l'utilisateur final. L'interface est composée de technologies standards (HTML/CSS/JS) qui communiquent avec le serveur via des protocoles HTTP (**Requêtes** / **Réponses**).
- **Couche Présentation (JSF)** : Cette couche gère l'affichage et l'interaction utilisateur. Elle se compose de :
 - **Pages Facelets (.xhtml)** : Les vues définies en XML qui génèrent le rendu final.
 - **Managed Beans (Contrôleurs CDI)** : Ils assurent le Binding de données et déclenchent les actions métier en réponse aux interactions sur la vue.
- **Couche Métier (Logique Application)** : C'est le cœur de l'application où sont centralisées les règles de gestion. Elle expose des **Services** (ex : **SubscriptionService**) qui traitent les demandes provenant de la couche présentation. Cette isolation permet de réutiliser la logique sans dépendre de l'interface graphique.

Architecture N-Tiers : VisionTech (JSF + Hibernate)

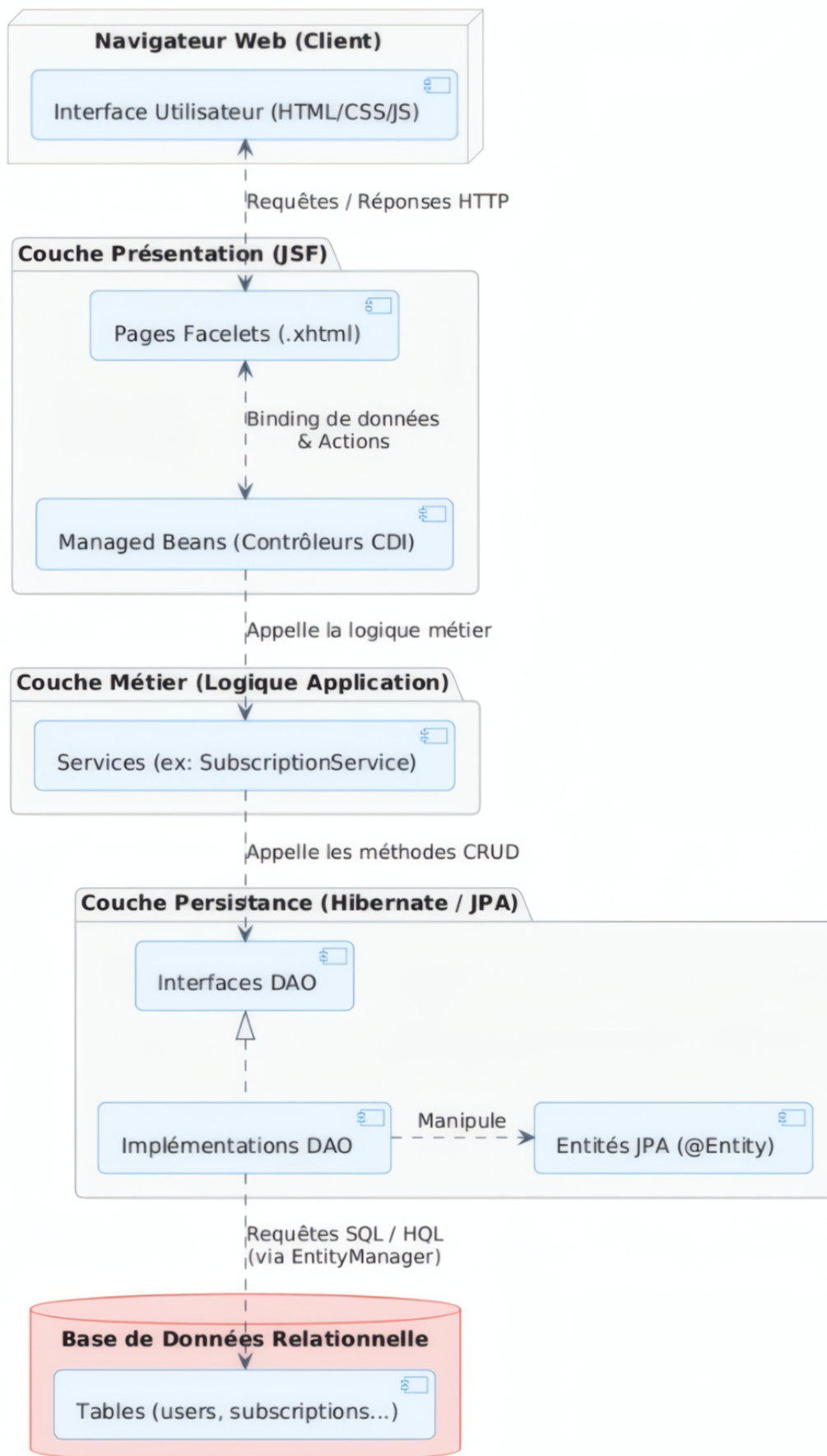


FIGURE 4 – Architecture N-Tiers de la plateforme VisionTech (JSF + Hibernate)

- **Couche Persistance (Hibernate / JPA) :** Ce module assure la traduction entre le monde objet et le monde relationnel (ORM).
 - **Interfaces et Implémentations DAO :** Le pattern *Data Access Object* est utilisé pour isoler les méthodes CRUD.
 - **Entités JPA (@Entity) :** Classes Java annotées qui représentent les données métier.
 - **EntityManager :** Gère les transactions et traduit les opérations en requêtes SQL / HQL vers la base de données.
- **Base de Données Relationnelle :** Le stockage physique des données (tables *users*, *subscriptions*, etc.) qui garantit l'intégrité et la persistance des informations de la plateforme.

9 Interfaces Principales de l'Application

Cette section illustre la réalisation concrète de l'application **SubTrack**. L'interface a été conçue pour être "Responsive" et intuitive, permettant une prise en main immédiate par l'utilisateur.

9.1 Pilotage et Analyse des Données

Les deux interfaces ci-dessous constituent le noyau décisionnel pour l'utilisateur. Elles permettent de transformer des données brutes en informations visuelles exploitables.

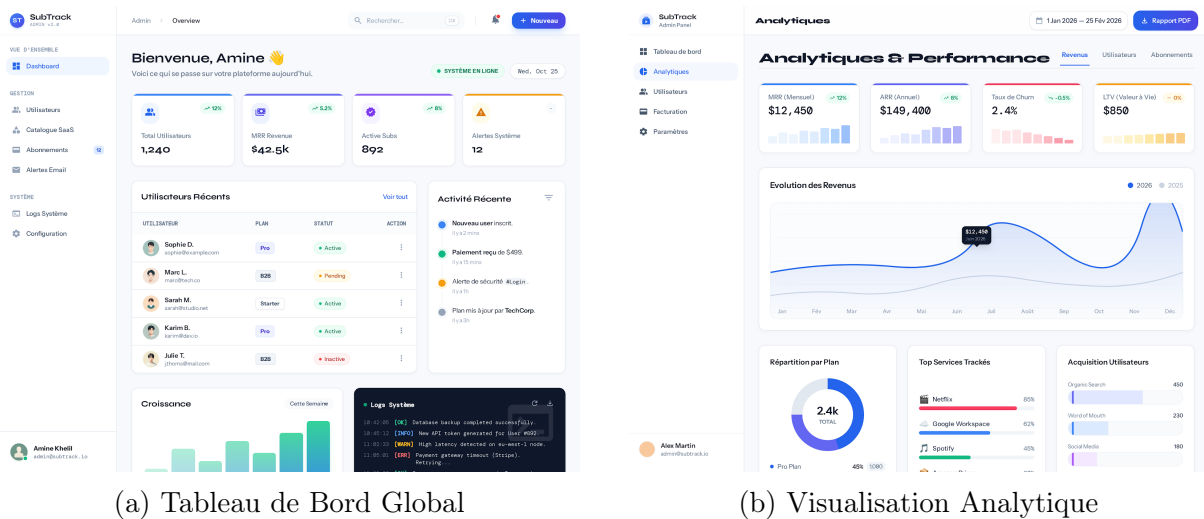


FIGURE 5 – Le Dashboard (Figure 5a) offre une vue d'ensemble, tandis que la page d'Analyse (Figure 5b) détaille les dépenses par catégorie.

9.2 Gestion du Catalogue SaaS

Le catalogue centralise l'ensemble des services disponibles sur le marché, permettant au client d'ajouter un service en quelques secondes grâce à des modèles pré-configurés.

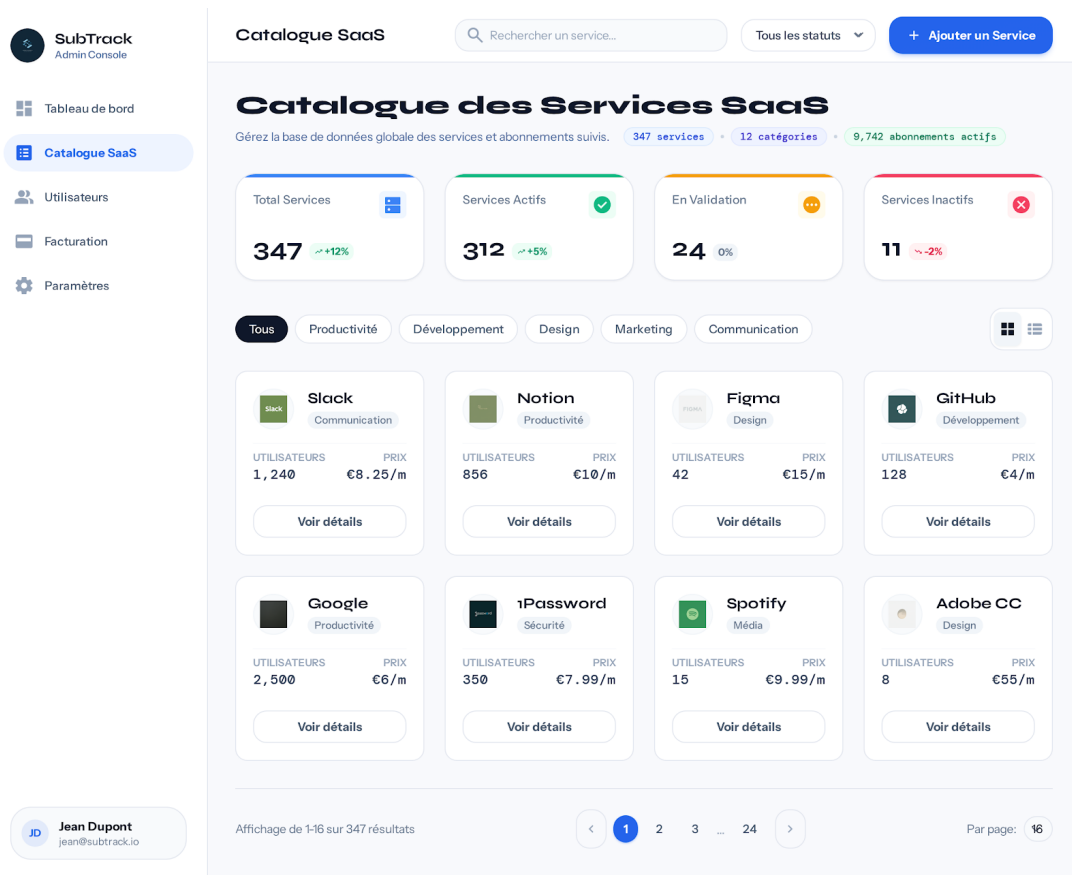
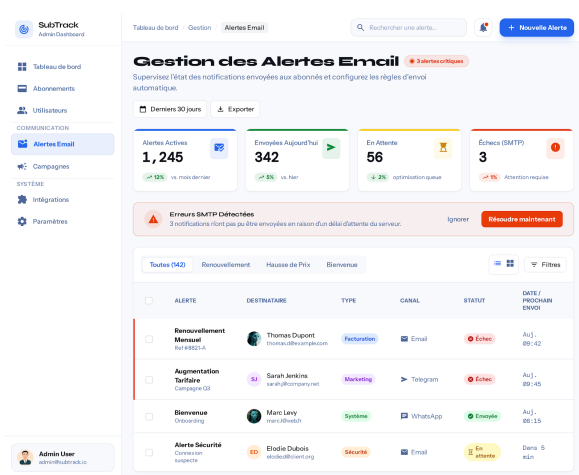


FIGURE 6 – Interface du Catalogue SaaS : Recherche et sélection de services.

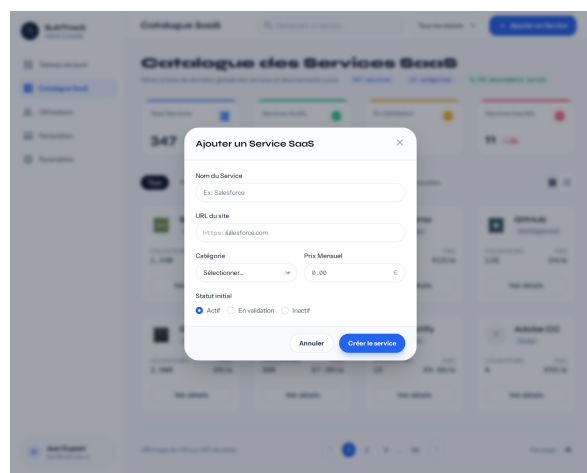
9.3 Configuration des Notifications et Gestion Profil

Afin d'éviter les surprises bancaires, l'utilisateur dispose d'un module d'alertes personnalisées et d'une page de gestion de ses informations.

- **Automatisation** : Toutes ces interfaces sont reliées dynamiquement à la base de données via les Managed Beans (JSF).
- **Expérience Utilisateur** : Les graphiques (Analyse) sont générés dynamiquement en fonction de l'historique de paiement stocké.



(a) Centre de Notifications



(b) Interface de Profil / Gestion

FIGURE 7 – Configuration des alertes (Figure 7a) et page principale de gestion (Figure 7b).